

智能系统设计实践

六足机器人避障控制结题汇报

谭诗弘，黄育民，曹洪琿

武汉大学电子信息学院

2025 年 11 月 3 日



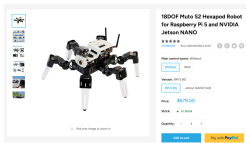
- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现
- 7 总结和展望

六足机器人业界现状

近年来，随着人工智能、机器人技术与自动控制理论的发展，仿生机器人逐渐成为研究热点。六足机器人凭借以下优势：

- 结构稳定性高
- 步态灵活多样
- 适应复杂地形能力强

在灾害救援、地质勘探、军事侦察等领域展现出广阔应用前景。与轮式或履带式机器人相比，六足机器人能够在崎岖不平的地形环境下保持稳定行走，这为其在野外环境中的任务执行提供了独特优势。



六足机器人避障的难点与研究动机

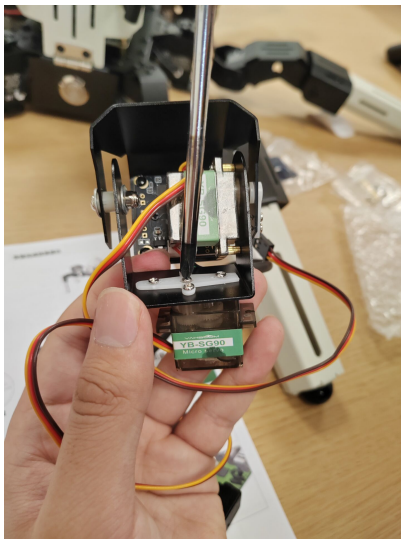
六足机器人在复杂环境下的自主避障，面临以下挑战：

- 姿态控制复杂：六足机器人具有多个自由度，在避障过程中需要同时兼顾步态稳定性、能耗优化和绕障效率。
- 视觉与控制耦合困难：视觉传感器获取的环境信息存在延迟、噪声和不确定性，如何与实时步态调整协调仍是难题。
- 环境动态性强：障碍物的形态与分布复杂，导致传统静态路径规划方法难以满足实时避障需求。

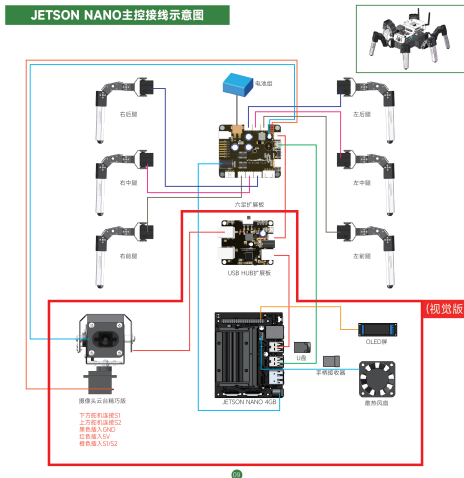
研究动机：本课题旨在探索六足机器人多足步态控制并实现避障的方法，提升其在复杂环境下的避障能力与自主性。

- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现
- 7 总结和展望

六足机器人组装测试



六足机器人硬件架构



六足机器人硬件架构

- 顶层 (Jetson Nano):
负责路径规划、目标识别、避障算法、通信等功能, 通过串口/总线将指令下发给 STM32 扩展板。Jetson Nano 采用带桌面的 Ubuntu 作为操作系统。
- 底层 (STM32 控制板):
负责 PWM 舵机驱动、步态切换、姿态保持、传感器数据读取等, 通过预设的步态算法执行任务。
- 单腿硬件:
每只腿包含 3 个舵机, 分别是: 肩关节 (控制腿向前/后/侧方向摆动, 决定运动方向 (前进、转向、横移)), 大腿关节 (控制大腿的抬起和放下) 和膝关节 (控制小腿弯曲/伸直, 影响步幅长度和对地形的适应性。))。

六足机器人软件架构

- 人机输入层（操作者）
输入设备：手柄 / 手机 APP / 键盘 / 上位机 GUI（带摄像头流/深度视图）
- 上位控制层（Jetson Nano）
接收人控指令、实时显示视频、将高层命令转为步态/动作模板 + 参数、发送到 STM32。使用 Python/ROS 节点（推荐 ROS 或 socket 通信）。
- 步态规划与变换层（Jetson Nano 或 STM32）
将“动作命令（如前进、左移、抬右前腿）”转为每条腿的期望足端轨迹（时序/位置/速度）。
- 逆运动学与舵机控制层（STM32）
把足端轨迹转为 18 个舵机角度，通过 PWM/串口控制舵机；做低级 PID 或 P 控制。

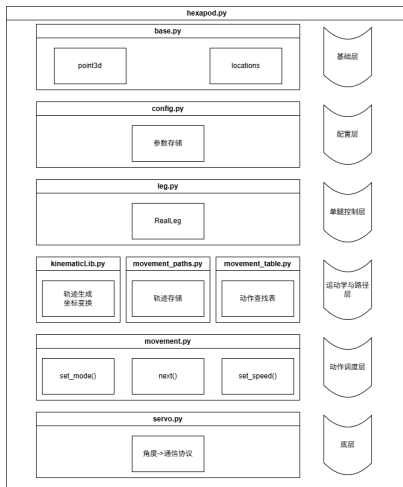
- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现
- 7 总结和展望

主要控制文件概览

共 11 个核心模块：

- ① base.py：基础坐标类与腿尖坐标系
- ② config.py：配置参数与预定义步态轨迹
- ③ hexapod.py：六足机器人主控制类
- ④ kinematicLib.py：运动学核心（逆运动学、轨迹生成、坐标变换）
- ⑤ leg.py：单腿控制类，负责逆运动学与舵机控制
- ⑥ movement_paths.py：预定义步态轨迹数据库
- ⑦ movement.py：高层动作控制器（模式切换、轨迹调度）
- ⑧ movement_table.py：静态动作路径表
- ⑨ MutoLib.py：机器人与上位机通信接口
- ⑩ pathPlanning.py：路径规划模块
- ⑪ servo.py：舵机控制模块

控制流程图



控制流程总结

- ① 用户输入（例如 `hexapod.move(10,0,0)` → 前进 10cm）。
- ② `movement.py` 根据输入调用 `movement_table` / `pathPlanning` 生成完整轨迹点。
- ③ `hexapod.py` 轮询调用 `movement.next()` → 取出 6 条腿的坐标。
- ④ `leg.py` 将目标点 → 逆运动学角度。
- ⑤ `servo.py` 把角度 → 舵机指令。
- ⑥ 舵机动作，机器人移动。

始终保持三条腿支撑、三条腿摆动的三角步态，从而保证平衡。

- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现**
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现
- 7 总结和展望

基于 JupyterLab 的基本运动代码实现和调试

- 在六足机器人上部署成功了 JupyterLab 环境，实现了直接板上运行代码和调试。
- 参考资料成功实现了在 Python 环境中控制六足机器人的运动。
- 实现了六足机器人的前进、后退、左转、右转等基本运动。
- 调用实现了六足机器人的表演动作。

代码实现-库导入和机器人定义

Muto 类负责整个机器人的协调，是机器人顶层类。

```
# 导入库 import library
import cv2
import time
import threading

import ipywidgets.widgets as widgets
from IPython.display import display
```

```
from MutoLib import Muto
g_bot = Muto()
```


代码实现-机器人控制

```
# 按键按下事件处理 Key press event processing
def on_button_clicked(b):
    with output:
        print("Button clicked:", b.description)
    if b.description == Name_widgets['Reset'][g_ENABLE_CHINESE]:
        g_bot.reset()
    elif b.description == Name_widgets['Stretch'][g_ENABLE_CHINESE]:
        g_bot.action(1)
    elif b.description == Name_widgets['Greeting'][g_ENABLE_CHINESE]:
        g_bot.action(2)
    elif b.description == Name_widgets['Retreat'][g_ENABLE_CHINESE]:
        g_bot.action(3)
    elif b.description == Name_widgets['Warm_up'][g_ENABLE_CHINESE]:
        g_bot.action(4)
    elif b.description == Name_widgets['Turn_around'][g_ENABLE_CHINESE]:
        g_bot.action(5)
    elif b.description == Name_widgets['Say_no'][g_ENABLE_CHINESE]:
        g_bot.action(6)
    elif b.description == Name_widgets['Crouching'][g_ENABLE_CHINESE]:
        g_bot.action(7)
    elif b.description == Name_widgets['Stride'][g_ENABLE_CHINESE]:
        g_bot.action(8)
```

```
def on_slider_step(value):
    global g_step
    g_step = value
    if button_forward.icon == 'check':
        g_bot.forward(g_step)
    elif button_backward.icon == 'check':
        g_bot.back(g_step)
    elif button_move_left.icon == 'check':
        g_bot.left(g_step)
    elif button_move_right.icon == 'check':
        g_bot.right(g_step)
    elif button_turn_left.icon == 'check':
        g_bot.turnleft(g_step)
    elif button_turn_right.icon == 'check':
        g_bot.turnright(g_step)
    print("step:", value)

widget_step = widgets.interactive(on_slider_step, value=slider_step)
```

代码实现-机器人控制

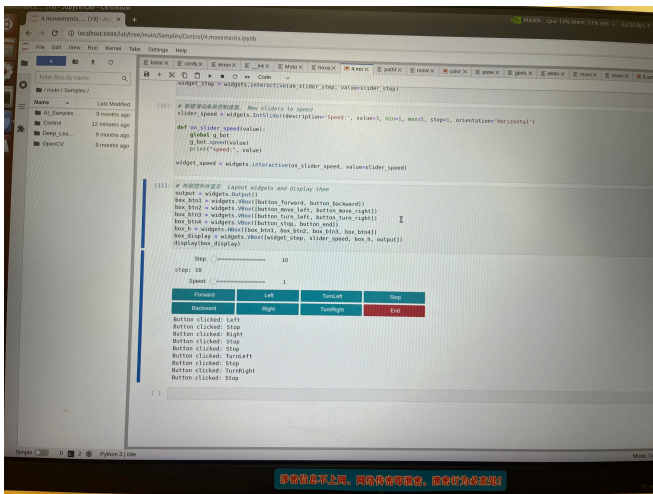
```
# 按键按下事件处理 Key press event processing
def on_button_clicked(b):
    with output:
        print("Button clicked:", b.description)
    if b.description == Name_widgets['Reset'][g_ENABLE_CHINESE]:
        g_bot.reset()
    elif b.description == Name_widgets['Stretch'][g_ENABLE_CHINESE]:
        g_bot.action(1)
    elif b.description == Name_widgets['Greeting'][g_ENABLE_CHINESE]:
        g_bot.action(2)
    elif b.description == Name_widgets['Retreat'][g_ENABLE_CHINESE]:
        g_bot.action(3)
    elif b.description == Name_widgets['Warm_up'][g_ENABLE_CHINESE]:
        g_bot.action(4)
    elif b.description == Name_widgets['Turn_around'][g_ENABLE_CHINESE]:
        g_bot.action(5)
    elif b.description == Name_widgets['Say_no'][g_ENABLE_CHINESE]:
        g_bot.action(6)
    elif b.description == Name_widgets['Crouching'][g_ENABLE_CHINESE]:
        g_bot.action(7)
    elif b.description == Name_widgets['Stride'][g_ENABLE_CHINESE]:
        g_bot.action(8)
```

```
def on_slider_step(value):
    global g_step
    g_step = value
    if button_forward.icon == 'check':
        g_bot.forward(g_step)
    elif button_backward.icon == 'check':
        g_bot.back(g_step)
    elif button_move_left.icon == 'check':
        g_bot.left(g_step)
    elif button_move_right.icon == 'check':
        g_bot.right(g_step)
    elif button_turn_left.icon == 'check':
        g_bot.turnleft(g_step)
    elif button_turn_right.icon == 'check':
        g_bot.turnright(g_step)
    print("step:", value)

widget_step = widgets.interactive(on_slider_step, value=slider_step)
```


实现效果-基础控制

使用滑块空间来控制机器人的步长和速度。



- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现
- 7 总结和展望

IMU 模块介绍

1.1 IMU 的组成

惯性测量单元（Inertial Measurement Unit，简称 IMU）通常由以下三种传感器组成：

传感器	测量内容	输出作用
加速度计（Accelerometer）	三轴线性加速度（X、Y、Z）	检测重力方向，估计倾斜角度
陀螺仪（Gyroscope）	三轴角速度（ ω_x 、 ω_y 、 ω_z ）	检测旋转速度，计算姿态变化
磁力计（Magnetometer，可选）	三轴地磁方向	校正航向角，防止漂移

表 1: IMU 模块的主要组成

因此，一个 **6 轴 IMU** = 加速度计 + 陀螺仪；一个 **9 轴 IMU** = 加速度计 + 陀螺仪 + 磁力计。

IMU 返回数据含义

名称	中文名	含义	单位	典型范围
roll	横滚角	绕 X 轴旋转的角度，表示机器人左右倾斜程度	°	-180° ~ +180°
pitch	俯仰角	绕 Y 轴旋转的角度，表示机器人前后倾斜程度	°	-90° ~ +90°
yaw	航向角	绕 Z 轴旋转的角度，表示机器人朝向（转弯方向）	°	0° ~ 360°
temp	温度	IMU 模块内部温度	°C	一般 20 ~ 40 °C

上坡与俯仰角 (pitch) 的关系

当机器人上坡或下坡时，主要导致机身前后倾斜，因此与 **pitch** (俯仰角) 直接相关。

- 上坡时：前方抬高 $\Rightarrow$ pitch 增大 (为正值)；
- 下坡时：前方降低 $\Rightarrow$ pitch 减小 (为负值)。

示例：

$$\text{imu} = [0.54, 1.45, 51.77, 20]$$

- roll = 0.54°：机器人基本水平，无明显左右倾斜；
- pitch = 1.45°：机器人略微前倾，可能处于轻微上坡；
- yaw = 51.77°：机器人当前朝向；
- temp = 20°C：IMU 模块温度。

```
imu = g_bot.read_IMU()
print("imu:", imu)
```

imu: [0.54, 1.45, 51.77, 20]

实验结果展示（二）

同样地，也可以通过 IMU 判断水平方向的倾斜：

```
[7]: imu = g_bot.read_IMU()
      print("imu:", imu)

      imu: [-1.84, -0.43, -8.51, 20]
```

```
1]: imu = g_bot.read_IMU()
      print("imu:", imu)

      imu: [-10.65, 1.52, -177.55, 20]
```

```
[12]: imu = g_bot.read_IMU()
        print("imu:", imu)

        imu: [-22.76, -6.97, 179.43, 20]
```

- 1 课题背景
- 2 前期研究
- 3 基础控制方案研究
- 4 基础行走功能实现
- 5 机器人板载 IMU 模块研究
- 6 避障功能实现**
- 7 总结和展望

代码分析

`motor(servo_id, angle, runtime=100)`: 控制机器人的十八个舵机转动。

参数 `servo_id` 表示要控制的舵机 ID 号，每个舵机 ID 号都不同，`servo_id` 的取值范围为 `[1, 18]`，分别对应机器人上的 18 个总线舵机。

参数 `angle` 表示要控制舵机运动到的角度，`angle` 的取值范围为 `[-90, 90]`。

参数 `runtime` 为可选参数，默认值为 100，表示舵机运动的时间，在有效范围内，数值越小，转动速度越快。

代码分析

Servo_torque_on (servo_id=0)：打开舵机扭矩力。

Servo_torque_off (servo_id=0)：关闭舵机扭矩力。

可选参数 servo_id 表示要控制的舵机 ID 号，取值范围为 [0, 18]，当 servo_id=0 时，表示控制所有舵机扭矩力打开或关闭；当 servo_id=[1, 18]，表示控制对应 ID 号舵机扭矩力打开或者关闭。

load_leg (leg)：打开某一条腿上的三个舵机的扭矩力

unload_leg (leg)：关闭某一条腿上的三个舵机的扭矩力

参数 leg 表示要控制的腿 ID 号，leg 的取值范围为 [1, 6]，分别表示机器人的六条腿。

控制逻辑

- 机器人前行至障碍物前；机器人通过调用内置前进步态逐步接近障碍物，当前腿距离障碍物达到预设阈值时停止进入越障状态。
- 前腿跨越障碍物；前腿（Leg 1 与 Leg 6）依次执行抬腿、跨越、落地动作。两条前腿均越过障碍物后，机器人前进，使中腿接近障碍物。
- 中腿跨越障碍物；中腿（Leg 2、Leg 5）动作逻辑与前腿一致，但为避免碰撞，需要更高的抬升角度。中腿在跨越过程中承担机体主要支撑，因此其动作更加平缓，并结合 IMU 姿态数据进行微调。
- 后腿跨越障碍物。当前四条腿均处于障碍物前方时，机器人进入后腿跨越阶段。后腿（Leg 3、Leg 4）由于机体重心已前移，跨越动作更为顺畅，依旧遵循抬腿—跨越—落地流程。

模块化封装

```
1  def cross_obstacle():
2      # 1. Approach the obstacle
3      approach_obstacle()
4
5      # 2. Lift and cross front legs
6      cross_front_legs(time)
7      g_bot.forward(time)
8
9      # 3. Lift and cross middle legs
10     cross_middle_legs(time)
11     g_bot.forward(time)
12
13     # 4. Lift and cross back legs
14     cross_back_legs(time)
15     g_bot.forward(time)
```

实践效果

在六足机器人平台上，我们对所提出的避障功能进行了系统性的实验验证。该方法适用于所有可近似为类长方体形状的障碍物，并在实验中验证了其在不同材料、尺寸与环境条件下的稳定性。为了确保算法具有通用性，我们设定了如下适用条件：

- 障碍物高度不得超过机器人腹部高度；
- 障碍物宽度不得超过机器人的标准步长；

实验中使用了泡沫块、木板箱体、塑料收纳箱、纸板箱等不同尺寸与材质的障碍物。测试环境包含光滑实验室地面、轻微不平整的水泥地等多种场景。需要强调的是，该方法不适用于坑洞、沟槽、落差、台阶、斜坡等凹陷或非凸形地形，此类场景由于缺乏明确的可见几何外形，当前的行走机制无法支持有效位移。

- ① 课题背景
- ② 前期研究
- ③ 基础控制方案研究
- ④ 基础行走功能实现
- ⑤ 机器人板载 IMU 模块研究
- ⑥ 避障功能实现
- ⑦ 总结和展望

应用场景

在应用层面，该避障功能可用于多种实际场景，包括但不限于：

- 室内服务机器人任务（越过家具、储物箱等低矮障碍）；
- 仓储与物流环境（越过货物堆叠体、隔断板等矩形障碍）；
- 农业巡检与自然场地任务（越过低矮灌木、石块、农机具等简易障碍物）；
- 用于教学与科研的六足机器人平台，验证腿式移动的环境适应性。

人员分工

本项目的成员分工为：

- 曹洪琿负责前端 UI 控件的调试实现与越障功能的实景测试；
- 黄育民负责后端机器人舵机控制交互的实现与设计越障动作。
- 谭诗弘负责相关文献资料的调研和驱动库阅读、PPT 和报告的撰写。

Thanks!